Retrieval-Augmented Generation

Baseline Retrieval vs. Cross-Encoder Re-Ranking

Param Modi | 2025201087

Code repository: https://github.com/iParamModi/lma-assn-1
Dataset (Catan rulebook PDF): Google Drive
Corpus: Catan base-game rulebook (rules + almanac), one PDF of 16 pages, chunked into 100 passages of about 635 characters each.
Stack: bi-encoder `all-MiniLM-L6-v2` (384-d) with a FAISS inner-product index; cross-encoder `ms-marco-MiniLM-L-6-v2`; answers from `Qwen2.5-1.5B-Instruct`, run locally with greedy decoding.
Protocol: the baseline retrieves the top 10 and sends its top 3 to the LLM; re-ranking re-scores the same 10 and sends its top 3. The prompt is identical for both, only the three passages differ.

1. Introduction

In a RAG system the final answer can only be as good as the passages the model is given. If the correct passage is not retrieved, even a strong model cannot answer correctly. This project studies that retrieval step by comparing two ways of choosing the passages, over the *same* chunks and the *same* prompt, so the only difference is how the passages are ranked.

I chose a board game rulebook as the corpus because a rulebook is full of small rules, exceptions, penalties and words like "not", "only" and "except". These are exactly the cases where a simple similarity search makes mistakes, which makes the benefit of re-ranking easy to see and measure.

2. Setup and Method

Corpus. One PDF, the Catan base-game rulebook (16 pages). I read it page by page and split each page into overlapping windows of about 700 characters with 120 characters of overlap, so that a rule and its condition usually stay in the same chunk. This gives **100 chunks**, each tagged with its page number for traceability.

Models. The bi-encoder `all-MiniLM-L6-v2` (384-d) turns each chunk into one vector. The vectors are unit length and stored in a FAISS inner-product index, so an inner-product search equals cosine similarity and runs in milliseconds. The cross-encoder `ms-marco-MiniLM-L-6-v2` takes one (query, chunk) pair and returns a single relevance score. The answers are written by `Qwen2.5-1.5B-Instruct`, run locally with greedy decoding so the output is deterministic and the two pipelines are compared fairly.

Two pipelines. *Pipeline 1 (baseline):* embed the query, take the top 10 chunks by cosine similarity, and send the top 3 to the model → Answer 1. *Pipeline 2 (re-ranking):* take the *same* 10 chunks, re-score each (query, chunk) pair with the cross-encoder, re-sort, and send the new top 3 → Answer 2. The prompt is identical and tells the model to answer only from the passages and to say so if the answer is absent. Because only the three passages differ, any change between Answer 1 and Answer 2 comes from retrieval alone.

3. Architecture: Bi-Encoder vs. Cross-Encoder

A **bi-encoder** encodes the query and each chunk *separately*; the two texts meet only as points in vector space, compared by cosine. This is fast and indexable, every chunk vector is computed once and reused, but it is not precise, because a chunk with similar words or topic can score high without actually answering the question.

A **cross-encoder** puts the query and one chunk *together* in the input, so self-attention runs across

all tokens of both texts. It can therefore judge negation, qualifiers and exceptions far better and returns one accurate score for the pair. Its cost is that it must run once per pair and builds no reusable index, so it can only re-rank a short shortlist, never the full corpus. The two-stage pipeline is designed to get the speed of the first and the accuracy of the second.

4. Failure Mode Analysis and Rank Shifts

I use four failure queries, one for each category in the task. In every case the key chunk starts *below* rank 3 in the baseline (so it is never sent to the model), and the cross-encoder lifts it into the top 3.

Query	Category	Chunk	Page	Base #	Re-rank #
Seven is rolled?	Keyword trap	23	5	6	1
(same)		63	11	8	2
(same)		64	11	4	3
Settlement next to another?	Exception	67	12	8	1
Give away cards as a gift?	Negation	37	7	5	1
(same)		79	14	7	2
How many points to win?	Buried fact	39	7	4	1

Table 1: Chunks moved from ranks 4–10 into the top 3 by the cross-encoder.

Why the baseline fails.

- *Seven rolled:* the top 3 were general production text, the token note "there is no 7", and a contents page, same words, wrong content. The real robber-and-discard rule sat at 4/6/8.
- *Settlement:* the generic "how to build a settlement" text ranked high and the Distance Rule that answers the question sat at rank 8.
- *Give away:* the question sounds positive but the rule is a "you may not" rule; the bi-encoder blurred the negative and returned generic trade text (rule at rank 5).
- *Points:* the exact number 10 is inside one small worked example, pushed down to rank 4 by passages that mention points in general.

5. Qualitative Comparison: Answer 1 vs. Answer 2

Seven is rolled (keyword trap). The baseline lacked the discard rule, so its answer is almost empty; re-ranking supplies the full rule.

Answer 1: baseline top 3

When a "7" is rolled, the robber token is activated.

Answer 2: re-ranked top 3

When a "7" is rolled, no player receives resources. Each player counts their cards; anyone holding more than 7 must discard half (round down). The robber then moves to another number token or the desert, and that hex produces nothing until the robber moves again.

Verdict: baseline left out an essential rule; re-ranking fixed it.

Give away cards (negation). The baseline answers "No" but for the *wrong* reason.

Answer 1: baseline top 3

No, but it explains with the wrong rule (the "a resource runs out in the bank" clause): a small hallucination.

Answer 2: re-ranked top 3

No, "You may not give away cards." (the exact rule).

Verdict: baseline misstated the rule; re-ranking gave the correct reason.

Settlement distance (exception). The chunk jumped from rank 8 to 1. Both answers reach the right conclusion, but the re-ranked one is grounded in the Distance Rule chunk.

Answer 1: baseline top 3

No, a settlement "can be placed on any unoccupied intersection, as long as the Distance Rule is observed."

Answer 2: re-ranked top 3

No, you must follow the Distance Rule. (The model also adds an unrelated line about upgrading to a city once all five settlements are built.)

Verdict: retrieval clearly improved; the final answer was already adequate.

Points to win (buried fact). The chunk jumped from rank 4 to 1, but the number 10 appears in several passages, so both answers are correct.

Answer 1: baseline top 3

To win, you need to reach 10 victory points on your turn.

Answer 2: re-ranked top 3

To win, you need 10 victory points.

Verdict: a genuine rank rescue with little change to the answer, because the fact is repeated.

Lesson. Re-ranking always recovers the correct chunk, but it changes the *answer* most when the key fact lives in only one place (seven rolled, give away) and least when the fact is repeated across the document (settlement, points).

6. Latency Profiling

Retrieval-only timing (the model call is identical for both pipelines), averaged over repeats on CPU.

Query	Baseline (ms)	Re-rank (ms)	Extra (ms)	Slowdown
Seven is rolled?	20.6	649.6	629.0	31.5×
Settlement next to another?	19.8	553.5	533.7	27.9×
Give away cards?	21.1	420.9	399.8	19.9×
How many points to win?	20.8	437.3	416.5	21.1×

Table 2: Baseline retrieval vs. the full re-ranking pipeline.

The baseline is ≈20 ms (one encode plus an index search). Re-ranking adds 0.4–0.65 s because the cross-encoder runs 10 forward passes on the CPU, about 20–32× slower in the retrieval step. In absolute terms it is still under one second, and very small next to the model's own answer time (≈1 minute per answer on this machine), so the extra accuracy is usually worth it.

7. Scaling Question

We do not run the cross-encoder on the whole collection from the start because it has no index: a score is defined only for a (query, chunk) pair read together, so scoring N chunks needs N forward passes *per query*. For my 100 chunks that is already ≈0.5 s; for a real corpus of millions of chunks it would take minutes to hours for a single query, infeasible. The bi-encoder embeds each chunk

once, offline, and then each query is one encode plus a fast search. So the practical design is two stages: the cheap bi-encoder cuts the corpus to a top-10 shortlist, and the costly cross-encoder re-scores only those 10 candidates, most of the accuracy at a small fraction of the cost.

8. Observations and Limitations

- **Corpus size.** With only 100 chunks, the top 10 already covers a large part of the document, so the baseline is often good on its own; the biggest *answer* changes appear when a fact lives in a single chunk. On a much larger corpus the baseline would miss more, and re-ranking would help more often, so these results are a conservative view of its benefit.
- **Score meaning.** Cross-encoder scores are raw relevance logits, not probabilities; they can be negative, and only their *order* is used. A chunk can have a low or even negative score and still be the best of the shortlist (as seen for the "seven is rolled" query).
- **Answer model.** The model is small (1.5B) and runs on CPU with greedy decoding, which keeps answers short and deterministic, good for a fair comparison, but it occasionally adds an unrelated sentence (seen in the settlement answer). A larger model would read the same passages but write cleaner answers.
- **Latency.** The numbers are CPU-bound and vary between runs. The cross-encoder cost (10 forward passes) would drop sharply on a GPU or with a smaller re-ranker, while the bi-encoder search stays almost free either way.
- **Chunking.** The 700/120 setting is a trade-off: smaller chunks give sharper matches but split a rule from its condition, while larger chunks keep rules together but mix topics. The overlap is what usually keeps a rule and its exception inside the same chunk.

9. Conclusion

Retrieval decides answer quality in a RAG system. The bi-encoder baseline is fast but misses the right chunk when the question has tricky wording, a negation or an exception. Adding a cross-encoder re-ranking step moved the correct chunk into the top 3 in every one of the four test queries, and clearly improved the final answer in the strongest cases, it added a missing rule and removed a wrong reason. The extra time cost is small compared to the language model, and the cross-encoder cannot scale to the whole corpus, so the two-stage "retrieve then re-rank" design is a good and practical choice.